



Pichulman Miguel Angel
Trabajo Práctico III: "Clean Code / Code Review "

Objetivos

- Aplicar principios de Clean Code.
- Mejorar legibilidad y mantenibilidad.
- Aplicar técnicas de refactoring seguro.
- Comprender cohesión y acoplamiento.
- Realizar revisiones profesionales de código.
- Desarrollar criterios técnicos de calidad.

Contexto

El Hospital Central incorporó recientemente un módulo de facturación médica desarrollado por distintos programadores a lo largo del tiempo.

Aunque el sistema funciona correctamente, el código presenta problemas de legibilidad, baja cohesión y alto acoplamiento, dificultando el mantenimiento y la evolución futura del software.

El objetivo de este trabajo práctico será mejorar la calidad del código, aplicar técnicas de refactoring seguro y realizar un proceso formal de code review.

Código Base

Se adjunta en un archivo.

Actividades

Parte A — Clean Code

Refactorizar el código aplicando principios de Clean Code.

Mejorar al menos:

- Nombres de clases, métodos y variables.
- Legibilidad general.
- Separación de responsabilidades.
- Cohesión interna.
- Reducción de acoplamiento.
- Manejo de errores.

Importante: El comportamiento funcional del sistema no debe modificarse.



Pichulman Miguel Angel

```
import java.util.List;

public class Facturacion {
    private static final double DESCUENTO_PARTICULAR = 0.5;
    private static final double DESCUENTO_OBRA_SOCIAL = 0.1;
    private static final double DESCUENTO_FIJO = 100;
    private static final int TIPO_DESCUENTO = 3;

    public double calcularTotal(List<Double> costosEstudios,
                                boolean esParticular,
                                boolean tieneObraSocial,
                                int tipoPaciente)
    {
        validarEntrada(costosEstudios);
        double total = calcularSubtotalEstudios(costosEstudios, esParticular);
        total = aplicarDescuentoObraSocial(total, tieneObraSocial);
        total = aplicarDescuentoTipoPaciente(total, tipoPaciente);

        return total;
    }

    private void validarEntrada(List<Double> costosEstudios) {
        if (costosEstudios == null) {
            throw new IllegalArgumentException("El costo estudio no puede ser nulo.");
        }
    }

    private double calcularSubtotalEstudios(List<Double> costosEstudios, boolean
esParticular) {
        double subbttotal = 0;
        for (Double costo : costosEstudios) {
            if (costo != null) {
                subbttotal += esParticular ? costo * DESCUENTO_PARTICULAR : costo;
            }
        }
        return subbttotal;
    }

    private double aplicarDescuentoObraSocial(double total, boolean tieneObraSocial) {
        if (tieneObraSocial) {
            return total * (1-DESCUENTO_OBRA_SOCIAL);
        }
        return total;
    }

    private double aplicarDescuentoTipoPaciente(double total, int tipoPaciente) {
        if (tipoPaciente > TIPO_DESCUENTO) {
            return total - DESCUENTO_FIJO;
        }
        return total;
    }
}
```

Pichulman Miguel Angel

Parte B — Refactoring Seguro

Aplicar técnicas de refactoring seguro sobre el código original.

Implementar obligatoriamente:

- Extract Method
- Introduce Parameter Object
- Move Method

Explicar brevemente:

- Qué problema resolvía cada técnica.
- Qué mejora aporta al diseño.

-Introduce Parameter Object

Problema que resolvía: El método `calc` recibía 4 parámetros primitivos y dispersos (`List<Double>` estudios, boolean `p`, boolean `o`, int `t`), aumentando la complejidad de la firma (`Data Clumps`).

Mejora al diseño: Agrupa los parámetros cohesivos en la clase `DatosFacturacion`, simplificando la firma de los métodos y facilitando la extensibilidad sin romper interfaces existentes.

-Extract Method

Problema que resolvía: En el código original, la iteración sobre la lista, el ajuste según particular y la lógica acumulativa estaban mezclados en un único bloque de código.

Mejora al diseño: Se extrae `calcularSubtotalEstudios`, otorgando un nombre representativo a la operación, aislando la iteración y facilitando la lectura directa del método principal.

-Move Method

Problema que resolvía: `Facturacion` concentraba tanto el cálculo base de estudios como las reglas de negocio de descuentos por tipo y obra social, violando la alta cohesión.

Mejora al diseño: Se crea `CalculadoraDescuentos` y se mueve hacia allí el método `aplicarDescuentos`, redistribuyendo responsabilidades y reduciendo el acoplamiento directo entre el flujo general y las reglas específicas de descuento.



Pichulman Miguel Angel

Parte C — Code Review

Realizar una revisión técnica del código utilizando una checklist profesional.

Checklist mínima obligatoria

- ¿Los nombres son descriptivos?
No. El método `calc` y las variables `p`, `o`, `t`, `e` y `r` utilizan identificadores de una sola letra o abreviaturas que no transmiten el contexto de negocio ni autodocumentan el código.
- ¿Las funciones poseen una única responsabilidad?
No. El método `calc` acumula múltiples niveles de abstracción: itera sobre la lista, calcula subtotales condicionales y aplica distintas reglas de descuento de forma secuencial.
- ¿Existe duplicación innecesaria?
Sí. En el bucle principal, la sentencia de acumulación `r += e` se repite dentro de la estructura condicional con una variación mínima que podría sintetizarse.
- ¿El código es fácilmente extensible?
No. Cualquier incorporación de una nueva regla impositiva o categoría de cliente requerirá modificar directamente el cuerpo del método existente, aumentando el riesgo de introducir regresiones.
- ¿Hay manejo correcto de errores?
No. No existen validaciones de entrada; si el parámetro `estudios` o alguno de sus elementos internos es `null`, el método lanzará una excepción no controlada (`NullPointerException`).
- ¿Existe acoplamiento innecesario?
Sí. Las reglas de cálculo y de descuentos están fuertemente acopladas dentro de un único método y clase, impidiendo reutilizarlas de forma independiente.
- ¿La cohesión es adecuada?
No. La clase `Facturacion` mezcla la orquestación general del cálculo con las políticas comerciales específicas y el control de flujo de la colección.
- ¿Se respetan principios SOLID?
No. Se vulnera principalmente el principio de responsabilidad única (SRP) y el principio de abierto/cerrado (OCP), ya que la clase debe modificarse internamente ante cualquier cambio de reglas.
- ¿El código resulta legible para otro desarrollador?
No. La presencia de nombres crípticos, comparaciones booleanas redundantes (`p == true`) y números mágicos sin nombre (`0.5`, `0.9`, `100`, `3`) dificulta la comprensión inmediata sin asistencia externa.

Pichulman Miguel Angel

Parte D — Feedback Profesional

Simular un proceso real de revisión entre desarrolladores.

Elaborar feedback técnico profesional sobre el código revisado.

Ejemplo de feedback esperado

La función presenta múltiples responsabilidades, lo que dificulta su mantenibilidad. Se recomienda separar el cálculo de descuentos y promociones en métodos independientes para mejorar cohesión y facilitar futuras extensiones.

Importante: El feedback debe ser técnico, objetivo y profesional. No deben utilizarse comentarios despectivos o personales.

FeedBack:

Buen trabajo estructurando la lógica de cálculo inicial de forma directa y compacta. Tras revisar la implementación, propongo las siguientes sugerencias técnicas para alinear el código con los estándares de *Clean Code* y mejorar su mantenibilidad:

- Legibilidad y nombres de dominio:

Motivo: Parámetros como `p`, `o`, `t` o el nombre del método `calc` resultan ambiguos y obligan a inferir el contexto de negocio.

Sugerencia: Renombrar a identificadores expresivos como `calcularTotal`, `esParticular`, `tieneObraSocial` y `tipoCliente`.

- Eliminación de números mágicos:

Motivo: Los valores literales `0.5`, `0.9`, `100` y `3` están codificados directamente en la lógica sin documentar su significado técnico o comercial.

Sugerencia: Extraerlos a constantes con nombres representativos (por ejemplo, `DESCUENTO_OBRA_SOCIAL`, `DESCUENTO_FIJO`, `TIPO_DESCUENTO`).

- Principio de Responsabilidad Única (SRP) y Cohesión:

Motivo: El método `calc` concentra la iteración de la colección, el cálculo base y la aplicación sucesiva de diferentes descuentos.

Sugerencia: Aplicar *Extract Method* para separar el cálculo del subtotal y la aplicación de los descuentos en métodos privados independientes o delegarlos a una clase especializada.

- Manejo seguro de errores:

Motivo: Si la lista `estudios` ingresa con un valor `null`, el código arrojará un `NullPointerException` en tiempo de ejecución.

Sugerencia: Incluir una validación temprana con `IllegalArgumentException` para garantizar una falla explícita y controlada.

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA



Pichulman Miguel Angel

- Simplificación de condicionales:

Motivo: Expresiones como `if (p == true)` resultan redundantes.

Sugerencia: Evaluar el booleano de forma directa (`if (esParticular)`).

Requisitos Técnicos

- Utilizar Java orientado a objetos.
- Aplicar buenas prácticas de Clean Code.
- Mantener compatibilidad funcional.
- Organizar correctamente clases y métodos.
- Entregar código legible y correctamente indentado.

Entregables

- Código fuente refactorizado.
- Documento PDF explicando las mejoras realizadas.
- Checklist de code review completa.
- Feedback profesional del code review.